

Week 2 — Python & GenAI Capstone Rubric

Multi-Agent RAG System for Enterprise Documentation

Overall Evaluation

[Exceeds Expectations / Meets Expectations / Needs Improvement]

Instructor's Qualitative Observations:

- [Rationale: Holistic review includes engagement, communication, and understanding, not just code.]
- Comment on:
 - Code quality and completeness
 - Trainee engagement and participation throughout the course
 - Demonstrated understanding (e.g., discussions, questions, or collaborative work)
 - Communication and professionalism
 - Key strengths and areas for growth

Evaluation Dimensions

This rubric evaluates the project across three key dimensions.

- To **Meet Expectations**, a participant must satisfy all baseline *Completion* criteria, and achieve "Meets Expectations" on all core *Concept Mastery* and *Process & Quality* items.
- **Exceeds Expectations** is awarded for advanced or comprehensive skills, especially in stretch-goal areas.

1. Completion (Baseline Requirement)

Status	Core Requirement
[Pass / Fail]	Runnable System: The project runs from a clean checkout using Python 3.8+. Dependencies are pinned and installed correctly; uses provided run scripts or configs.
[Pass / Fail]	End-to-End Query Handling: The system can be started via a CLI, accept a user query, and return a complete answer from at least two of the three required agents.

Milestone Completion

For each milestone below, indicate completion status (✓ / ✗) or partial points as appropriate.

Milestone	Status (✓ / ✗ / Partial)	Comments
Environment/setup complete		
Manager (orchestration) agent functional		
Qualitative (RAG) agent functional		
Quantitative (NL-to-SQL) agent functional		
Multi-agent integration (all agents work)		
CLI interface operational		
Automated testing (basic or partial)		
Documentation draft (README, diagram)		
Total:	X/8	

If either "Runnable System" or "End-to-End Query Handling" is marked **Fail, the project is considered "Needs Improvement" overall.*

2. Concept Mastery

Needs Improvement / Meets Expectations / Exceeds Expectations	Key Concept	Instructor Notes
	Agent Orchestration (Manager)	Manager agent correctly classifies queries, routes to the right agent, and coordinates final responses. Handles unsupported queries gracefully.(e.g., <i>Routing logic is effective for simple queries, struggles with complex ones.</i>)
	Qualitative RAG Agent	Integrates with a vector DB and LLM to retrieve document chunks, with accurate citations/URLs.(e.g., <i>Attribution is excellent; could improve chunking strategy.</i>)
	Quantitative NL-to-SQL Agent	Connects to a SQL DB, translates natural language to SQL, executes, and returns a tabular/numeric answer.(e.g., <i>Handles single-table queries, but struggles with joins.</i>)
	Full Three-Agent Integration (Stretch)	All three agents work seamlessly; the system handles all query types without errors.(e.g., <i>Fully robust, production-ready architecture.</i>)
	GenAI Fundamentals & Mental Models	[Rationale: Captures understanding of LLMs, agent-tool loops, prompt engineering, and system limitations.]Does the participant demonstrate foundational understanding of LLM behavior (e.g., hallucination, tokenization, prompt context), agent-tool interaction patterns, and the strengths/limitations of GenAI approaches? (e.g., <i>Can explain why a prompt might fail, or how tool-calling agents differ from simple LLM chatbots.</i>)

3. Process & Quality

Needs Improvement / Meets Expectations / Exceeds Expectations	Engineering Practice	Instructor Notes (examples)
	Documentation (README)	README covers system architecture, agent roles, step-by-step setup, and usage. Diagram illustrates agent interaction.(e.g., <i>Setup is clear, but diagram could be added.</i>)
	Command-Line Interface	CLI is user-friendly, gives clear prompts, formatted output, and clean exit.(e.g., <i>Functional, but could handle invalid input better.</i>)
	Automated Testing	[Rationale: Clarifies realistic expectations for most learners.]Partial coverage is acceptable at “Meets”; comprehensive suite required only for “Exceeds.” Tests may cover agents, integration, CLI.(e.g., <i>Good agent-level unit tests; limited integration tests.</i>)
	Code Quality & Pythonic Style	[Rationale: Professional code standards for maintainability and readability.]Code follows idiomatic Python practices, is readable, maintainable, and would be easy for another developer to extend. Naming, modularity, and clarity are prioritized.(e.g., <i>Clean abstractions, consistent style, minimal tech debt.</i>)